

3. Async/await

3.1. Async y await

Trabajar con promesas simplifica el trabajo con código asíncrono. Aparte, JavaScript define un mecanismo complementario: **async/await**.

async es un modificador que se utiliza en la definición de una función para indicar que dicha función es asíncrona y que por tanto va a devolver una promesa. Así, el resultado de la función siempre será un objeto promesa, por lo que tendrá acceso a **then** y **catch**. Si la función devuelve un valor normal, dicho valor será convertido a una promesa que se resuelve de manera inmediata. Por ejemplo:

```
async function multiplicar(a,b) {  
    return a*b;  
}  
  
let a = multiplicar(3,5);  
  
console.log(a); // Objeto promesa  
  
a.then(resultado => console.log(resultado)); //15
```

En el ejemplo anterior, **multiplicar** es una función asíncrona, por lo que su resultado es una promesa. Así, la variable **a** almacena una promesa cuyo resultado es accesible dentro del método **then**.

await es una palabra reservada que se utiliza para esperar a que una promesa se resuelva y obtener el resultado. Solo puede utilizarse dentro de funciones definidas como **async**. Por ejemplo:

```
async function programaPrincipal() {  
  
    // La función 'temporizador' devuelve una promesa  
    function temporizador(tiempo) {  
        return new Promise(function(resolver, rechazar) {  
            setTimeout(function() {  
                resolver(`Temporizador de ${tiempo} ms terminado`);  
            }, tiempo);  
        });  
    }  
  
    // Se puede utilizar 'await' porque estamos dentro de la función  
    'programaPrincipal',  
    // que está definida como 'async'  
    let a = await temporizador(3000);  
    // El programa espera a que se resuelva la promesa y se obtenga el  
    resultado  
  
    // La variable 'a' almacena el resultado, NO LA PROMESA  
    console.log(a); // Temporizador de 3000 ms terminado  
}  
  
// Se ejecuta el programa principal  
programaPrincipal();  
// El programa espera 3 segundos...  
// Y a los 3 segundos aparece el mensaje
```

En el programa anterior se define una función `programaPrincipal` como función `async`. Dentro de dicha función, se espera a que termine la función `temporizador(3000)`, y cuando termina se muestra su resultado. Como puede comprobarse, la variable `a` en este caso no almacena una promesa, sino que almacena el resultado de la ejecución de la promesa.

! IMPORTANTE

Recuerda: solo puedes utilizar `await` dentro de funciones definidas como `async`.

También puedes utilizar `await` directamente dentro de módulos sin necesidad de tener que estar dentro en una función `async`.

Mediante la utilización de `await` se puede utilizar código asíncrono como si fuera código `síncrono`. El código que se define a continuación de una llamada a una función con `await` no se ejecuta hasta que dicha función finalice. Comparemos, por ejemplo, esta versión de código:

```
// Función asíncrona
miFuncionAsincrona().then(function(a) {
    // Esta sentencia se ejecuta cuando termina 'miFuncionAsincrona()'
    // Se ejecuta DESPUÉS de que aparezca el texto 'Hola mundo'
    console.log(a);
});

// Esta sentencia se ejecuta inmediatamente después de la sentencia
anterior
// NO ESPERA a que 'miFuncionAsincrona()' termine y devuelva un
resultado
console.log("Hola mundo");
```

Con esta otra:

```
async function programaPrincipal() {
    // Llamada a una función con 'await'
    let a = await miFuncionAsincrona();

    // Esta sentencia no se ejecuta hasta que termina la sentencia
anterior
    console.log(a);
}
```

Como se puede ver, la segunda versión es más intuitiva de comprender: el código que aparece después se ejecuta después. Sin embargo, hay que tener en cuenta un detalle muy importante: si dentro de una función se realiza una llamada a `await`, se suspende la ejecución de dicha función hasta que la promesa se resuelve o se rechaza, pero la aplicación no queda bloqueada (por ejemplo, puede seguir respondiendo a eventos).

3.2. Tratamiento de errores con *async/await*

Tal como hemos comentado antes, **async/await** permite utilizar código asíncrono como si fuera código síncrono. Por ello, para el tratamiento de errores se debe utilizar el bloque **try...catch** convencional. Así, si la promesa que estamos esperando con **await** se rechaza, se genera un error que podemos capturar en el bloque **catch**. Por ejemplo:

```
async function programaPrincipal() {
  try {
    let a = await miFuncionAsincrona();
  } catch (error) {
    // Procesamiento del error si la promesa que devuelve
    'miFuncionAsincrona' se rechaza
  }
}
```

3.3. Promesas frente a *async/await*

Tanto las promesas como **async/await** permiten trabajar con código asíncrono. Es posible combinar su uso u optar por una u otra opción atendiendo a las siguientes restricciones y recomendaciones:

- **await** solo puede utilizarse en módulos y dentro de funciones definidas como **async**. Las promesas y sus métodos **then** y **catch** pueden utilizarse en cualquier tipo de función, incluido el programa principal.
- Es posible combinar el uso de **await** y los métodos **then** y **catch** de las promesas. Aun así, es recomendable optar por una u otra alternativa, por motivos de coherencia y legibilidad del código.

Por ejemplo, el siguiente código utiliza una combinación de **await** y promesas (la función **fetch** es una función asíncrona que estudiaremos en el siguiente apartado de la unidad):

```
async function programaPrincipal() {
  // 'fetch()' devuelve una promesa
  let respuesta = await fetch("https://api.github.com/users/vuejs");
  // 'json()' devuelve otra promesa
  respuesta.json().then(datos => {
    console.log(datos.login);
  });
}

programaPrincipal();
```

Es más recomendable utilizar solo **await**:

```
async function programaPrincipal() {
  let respuesta = await fetch("https://api.github.com/users/vuejs");
  let datos = await respuesta.json();
  console.log(datos.login);
}

programaPrincipal();
```

O solo promesas:

```
function programaPrincipal() {  
  fetch("https://api.github.com/users/vuejs").then(respuesta => {  
    return respuesta.json();  
  }).then(datos => {  
    console.log(datos.login);  
  });  
}  
  
programaPrincipal();
```